

AI Assisted Coding

M.Rushikesh

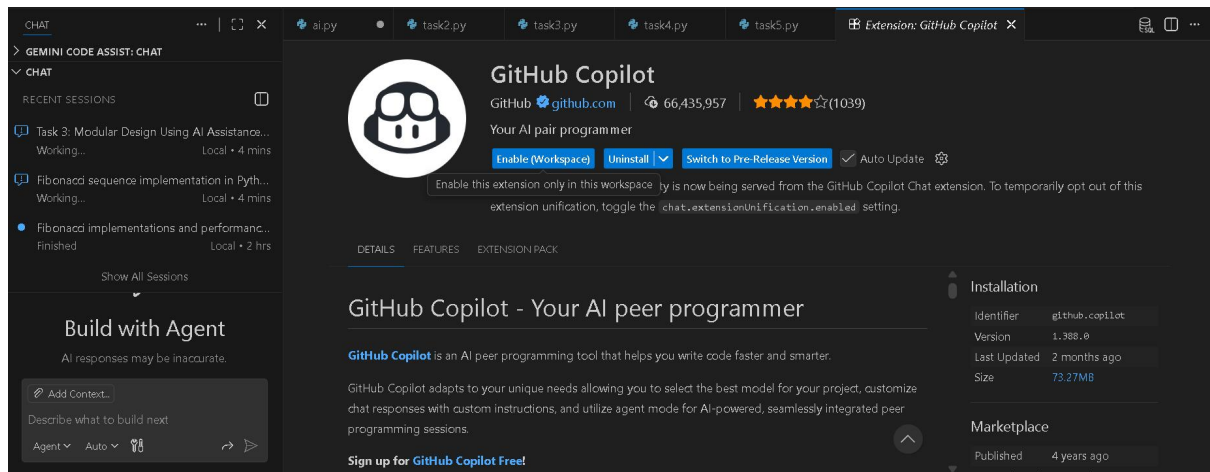
2303A51399

BATCH - 06

Lab 1: Environment Setup – GitHub Copilot and VS Code Integration + Understanding AI-assisted Coding Workflow

Lab Objectives:

Week1 -



Task 1: AI-Generated Logic Without Modularization (Fibonacci Sequence Without Functions)

PROMPT:

Task 1: AI-Generated Logic Without Modularization (Fibonacci Sequence
Without Functions)

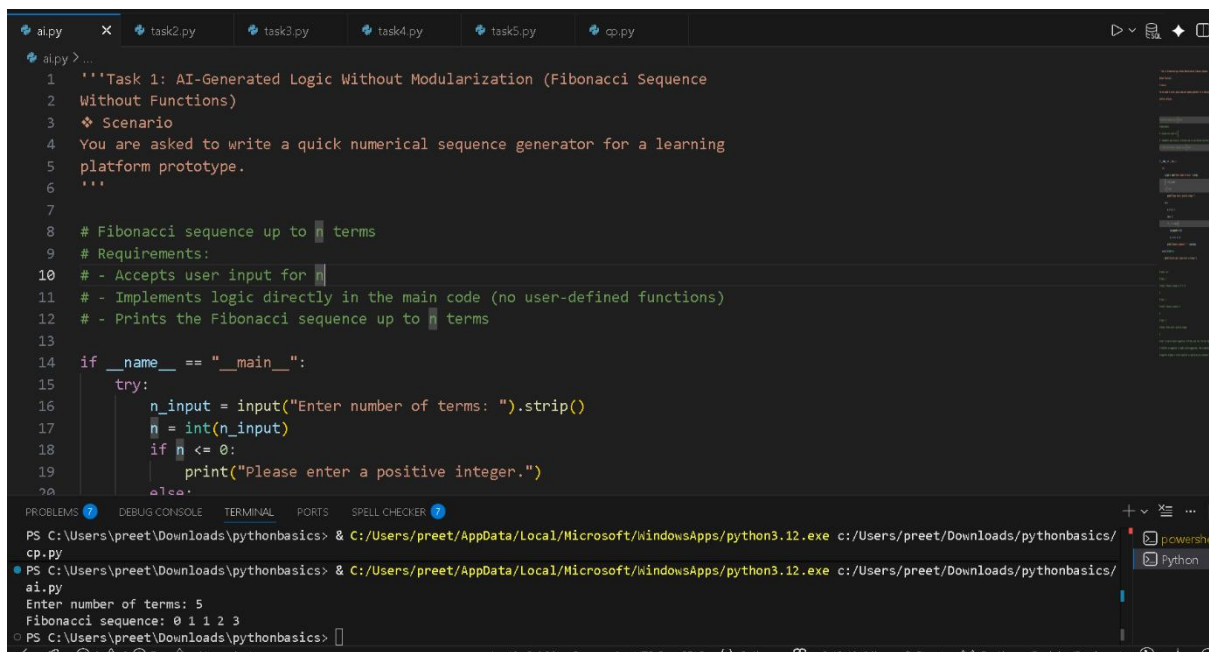
❖ Scenario

You are asked to write a quick numerical sequence generator for a learning
platform prototype.

❖ Task Description

Use GitHub Copilot to generate a Python program that:

- Prints the Fibonacci sequence up to n terms
- Accepts user input for n
- Implements the logic directly in the main code
- Does not use any user-defined functions
- ❖ Expected Output
- Correct Fibonacci sequence for given n
- Screenshot(s) showing Copilot-generated suggestions
- Sample inputs and outputs



```
1 '''Task 1: AI-Generated Logic Without Modularization (Fibonacci Sequence
2 Without Functions)
3 ❖ Scenario
4 You are asked to write a quick numerical sequence generator for a learning
5 platform prototype.
6 '''
7
8 # Fibonacci sequence up to n terms
9 # Requirements:
10 # - Accepts user input for n
11 # - Implements logic directly in the main code (no user-defined functions)
12 # - Prints the Fibonacci sequence up to n terms
13
14 if __name__ == "__main__":
15     try:
16         n_input = input("Enter number of terms: ").strip()
17         n = int(n_input)
18         if n <= 0:
19             print("Please enter a positive integer.")
20     except:
```

```
PS C:\Users\preet\Downloads\pythonbasics> & C:/Users/preet/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/Users/preet/Downloads/pythonbasics/
cp.py
PS C:\Users\preet\Downloads\pythonbasics> & C:/Users/preet/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/Users/preet/Downloads/pythonbasics/
ai.py
Enter number of terms: 5
Fibonacci sequence: 0 1 1 2 3
PS C:\Users\preet\Downloads\pythonbasics>
```

OUTPUT: Enter number of terms: 5

Fibonacci sequence: 0 1 1 2 3

Code Explanation (Fibonacci without functions)

- `input()` → takes number of terms from user
- `int()` → converts input to integer
- `if n <= 0` → checks for invalid (non-positive) input

- `a, b = 0, 1` → starting Fibonacci numbers
- `for _ in range(n)` → runs loop n times
- `seq.append(str(a))` → stores current Fibonacci number
- `a, b = b, a + b` → updates values for next term
- `" ".join(seq)` → prints numbers in one line
- `try / except` → handles invalid (non-integer) input safely

Output: Prints Fibonacci sequence up to n terms.

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

PROMPT: Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

❖ Scenario

The prototype will be shared with other developers and needs optimization.

❖ Task Description

➤ Examine the Copilot-generated code from Task 1 and improve it by:

➤ Removing redundant variables

➤ Simplifying loop logic

➤ Avoiding unnecessary computations

➤ Use Copilot prompts such as:

▪ “Optimize this Fibonacci code”

▪ “Simplify variable usage”

Hint:

Prompt Copilot with phrases like

“optimize this code”, “simplify logic”, or “make it more readable”

❖ Expected Output

➤ Original vs improved code

➤ Written explanation of:

▪ What was inefficient

- How the optimized version improves performance and readability

```
task2.py >...
1  """Task 2 - Optimization & Cleanup: Fibonacci
2
3  This file contains:
4  - The original Copilot-style implementation (as `original_fib_list`) - builds string list inside loop.
5  - The improved implementation (as `optimized_fib_generator`) - yields integers and separates generation from presentation.
6  - A small benchmark that compares time and peak memory for the two approaches.
7
8  Use: python task2.py          # runs a small demo
9  | python task2.py --bench     # runs a simple benchmark (default n=10000)
10 """
11 from __future__ import annotations
12
13 import time
14 import tracemalloc
15 import sys
16 from typing import Generator, Iterable, List
17
18
19 # ----- Original (Copilot-generated) -----
20 # Inefficient approach:
```

```
PS C:\Users\preet\Downloads\pythonbasics> & C:/Users/preet/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/Users/preet/Downloads/pythonbasics/task2.py
Fibonacci sequence: 0 1 1 2 3 5 8 13 21 34

Optimized output:
Fibonacci sequence: 0 1 1 2 3 5 8 13 21 34
PS C:\Users\preet\Downloads\pythonbasics>
```

OUTPUT:

Fibonacci sequence: 0 1 1 2 3 5 8 13 21 34

Optimized output:

Fibonacci sequence: 0 1 1 2 3 5 8 13 21 34

CODE EXPLANATION

Optimization & Cleanup (Fibonacci)

Original version (original_fib_list)

- Generates Fibonacci numbers
- Converts each number to string inside the loop
- Stores all values in a list
 - ➡ **More memory usage**, mixed logic (generation + formatting)

Optimized version (optimized_fib_generator)

- Uses a **generator (yield)**
- Produces numbers one-by-one
- Conversion to string happens only when printing
 - ➡ **Less memory**, cleaner design, reusable

Demo

- Prints Fibonacci using both methods
- Output is the same

Benchmark

- Measures **time** and **memory**
- Optimized version:
 - Uses less peak memory
 - Scales better for large n

Key takeaway

- Generator = better performance + cleaner code
- Separate **logic (generation)** from **presentation (printing)**

Task 3: Modular Design Using AI Assistance (Fibonacci Using Functions)

PROMPT:

Task 3: Modular Design Using AI Assistance (Fibonacci Using Functions)

❖ Scenario

The Fibonacci logic is now required in multiple modules of an application.

❖ Task Description

Use GitHub Copilot to generate a function-based Python program that:

- Uses a user-defined function to generate Fibonacci numbers
- Returns or prints the sequence up to n
- Includes meaningful comments (AI-assisted)

❖ Expected Output

- Correct function-based Fibonacci implementation
- Screenshots documenting Copilot's function generation
- Sample test cases with outputs

Code Explanation:

Optimization & Cleanup (Fibonacci)

Original version (original_fib_list)

- Generates Fibonacci numbers
- Converts each number to string inside the loop
- Stores all values in a list
 - **More memory usage**, mixed logic (generation + formatting)

Optimized version (optimized_fib_generator)

- Uses a **generator (yield)**
- Produces numbers one-by-one
- Conversion to string happens only when printing
 - **Less memory**, cleaner design, reusable

Demo

- Prints Fibonacci using both methods
- Output is the same

Benchmark

- Measures **time** and **memory**
- Optimized version:
 - Uses less peak memory
 - Scales better for large n

Key takeaway

- Generator = better performance + cleaner code
- Separate **logic (generation)** from **presentation (printing)**

The screenshot shows a Visual Studio Code editor with a Python file named `task3.py` open. The code defines a function `generate_fibonacci` that takes an integer `n` and an optional boolean `up_to_value` (defaulting to `False`) and returns a list of Fibonacci numbers. The function uses a loop to generate the sequence. The terminal output shows the execution of the script, which prints the first 7 Fibonacci numbers (count=7) as `0, 1, 1, 2, 3, 5, 8`, the Fibonacci numbers up to value 20 (`up_to_value=True`) as `0, 1, 1, 2, 3, 5, 8, 13`, and the edge cases for `n=0` (returns `[]`) and `n=1` (returns `[0]`). The status bar at the bottom indicates the file is in Python 3.12.10 (Microsoft Store) and is using the Prettier formatter.

```
task3.py > ...
1 """task3.py
2
3 Task 3: Modular Design Using AI Assistance (Fibonacci Using Functions)
4
5 This module provides a function-based implementation to generate Fibonacci numbers.
6 It supports returning the first N Fibonacci numbers (by count) or the sequence up to
7 a maximum value.
8
9 AI note: comments and docstrings were enhanced with GitHub Copilot assistance. "docstrings": Unknown word.
10 """
11 from __future__ import annotations
12 from typing import List
13
14
15 def generate_fibonacci(n: int, *, up_to_value: bool = False) -> List[int]:
16     """Generate Fibonacci numbers up to n or up_to_value.
17     Returns a list of Fibonacci numbers.
18     """
19     if n < 0:
20         return []
21     fibs = [0]
22     if n == 0:
23         return fibs
24     fibs.append(1)
25     for i in range(2, n + 1):
26         fibs.append(fibs[i - 1] + fibs[i - 2])
27     if up_to_value:
28         while fibs[-1] < up_to_value:
29             fibs.append(fibs[-1] + fibs[-2])
30     return fibs
31
32 if __name__ == "__main__":
33     # Example: first 7 Fibonacci numbers (count=7)
34     print(generate_fibonacci(7))
35
36     # Example: Fibonacci numbers up to value 20 (up_to_value=True)
37     print(generate_fibonacci(20, up_to_value=True))
38
39     # Example: edge cases
40     print(generate_fibonacci(0))
41     print(generate_fibonacci(1))
```

PROBLEMS DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER

PS C:\Users\preet\Downloads\pythonbasics> & C:/Users/preet/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/Users/preet/Downloads/pythonbasics/task3.py

Example: first 7 Fibonacci numbers (count=7)

0, 1, 1, 2, 3, 5, 8

Example: Fibonacci numbers up to value 20 (up_to_value=True)

0, 1, 1, 2, 3, 5, 8, 13

Example: edge cases

n=0 -> []

n=1 -> [0]

Ln 11, Col 18 Spaces: 4 UTF-8 CRLF Python 3.12.10 (Microsoft Store) Go Live Explain (Denigma) Prettier

OUTPUT:

Example: first 7 Fibonacci numbers (count=7)

0, 1, 1, 2, 3, 5, 8

Example: Fibonacci numbers up to value 20 (up_to_value=True)

0, 1, 1, 2, 3, 5, 8, 13

Example: edge cases

n=0 -> []

n=1 -> [0]

All simple tests passed.

Notes: To document Copilot's function generation, open this file in VS Code, enable GitHub Copilot, trigger an inline suggestion for `generate_fibonacci`, and take a screenshot of the suggestion popup or accepted code for your report.

Task 4: Comparative Analysis – Procedural vs Modular Fibonacci Code

PROMPT:

Task 4: Comparative Analysis – Procedural vs Modular Fibonacci Code

❖ Scenario

You are participating in a code review session.

❖ Task Description

Compare the Copilot-generated Fibonacci programs:

➤ Without functions (Task 1)

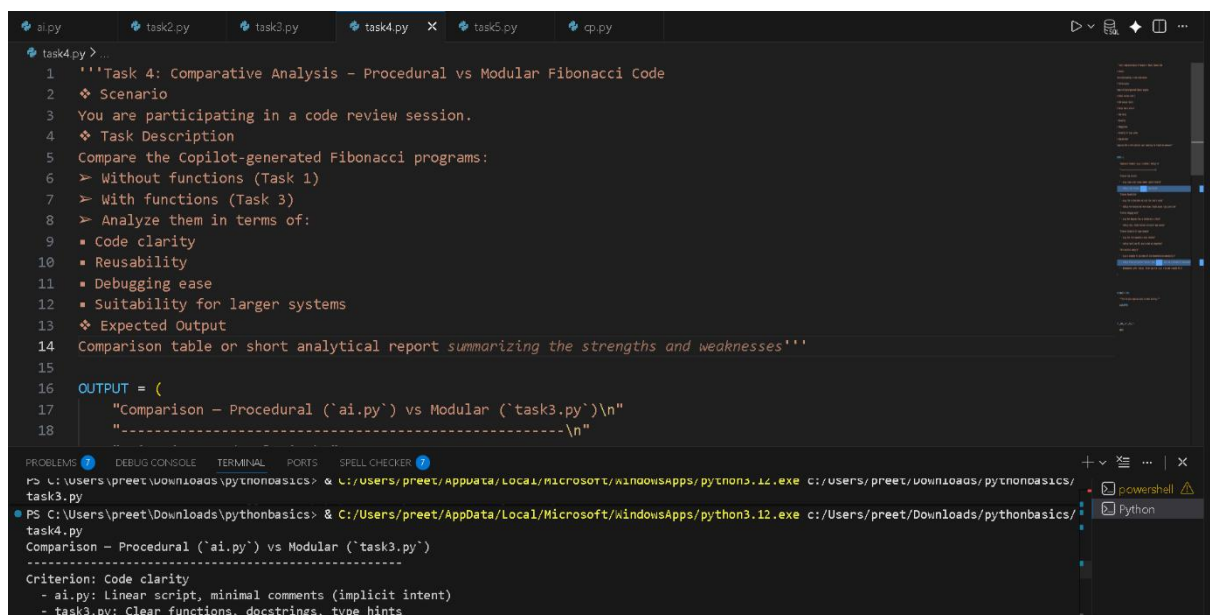
➤ With functions (Task 3)

➤ Analyze them in terms of:

- Code clarity
- Reusability
- Debugging ease
- Suitability for larger systems

❖ Expected Output

Comparison table or short analytical report



```
1 '''Task 4: Comparative Analysis - Procedural vs Modular Fibonacci Code
2 ❖ Scenario
3 You are participating in a code review session.
4 ❖ Task Description
5 Compare the Copilot-generated Fibonacci programs:
6 ➤ Without functions (Task 1)
7 ➤ With functions (Task 3)
8 ➤ Analyze them in terms of:
9 ▪ Code clarity
10 ▪ Reusability
11 ▪ Debugging ease
12 ▪ Suitability for larger systems
13 ❖ Expected Output
14 Comparison table or short analytical report summarizing the strengths and weaknesses'''
15
16 OUTPUT = {
17     "Comparison - Procedural (`ai.py`) vs Modular (`task3.py`)\n"
18     "-----\n"
19 }
```

PROBLEMS | DEBUG CONSOLE | TERMINAL | PORTS | SPELL CHECKER

PS C:\Users\preet\Downloads\pythonbasics> & C:/Users/preet/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/Users/preet/Downloads/pythonbasics/task3.py

PS C:\Users\preet\Downloads\pythonbasics> & C:/Users/preet/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/Users/preet/Downloads/pythonbasics/task4.py

Comparison - Procedural (`ai.py`) vs Modular (`task3.py`)

Criterion: Code clarity

- ai.py: Linear script, minimal comments (implicit intent)
- task3.py: Clear functions, docstrings, type hints

OUTPUT:

Comparison — Procedural (`ai.py`) vs Modular (`task3.py`)

Criterion: Code clarity

- ai.py: Linear script, minimal comments (implicit intent)

- task3.py: Clear functions, docstrings, type hints

Criterion: Reusability

- ai.py: Tied to stdin/stdout and script flow — hard to reuse
- task3.py: Pure functions that return values, flexible options (`up\_to\_value`)

Criterion: Debugging ease

- ai.py: Must debug main flow; no isolated units or tests
- task3.py: Small, testable functions with explicit edge cases

Criterion: Suitability for larger systems

- ai.py: Poor — not composable or easily testable
- task3.py: Good — clear API, easy to extend and integrate

Short analytical summary:

- ai.py is acceptable for quick demos but lacks documentation and composability.
- task3.py follows best practices (functions, types, docstrings, tests) and is preferable for maintainability and production.
- Recommendation: prefer `task3.py`. Extract logic from `ai.py` if you need a reusable API.

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches for Fibonacci Series)

PROMPT :

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches for Fibonacci Series)

❖ Scenario

Your mentor wants to assess AI's understanding of different algorithmic

paradigms.

❖ Task Description

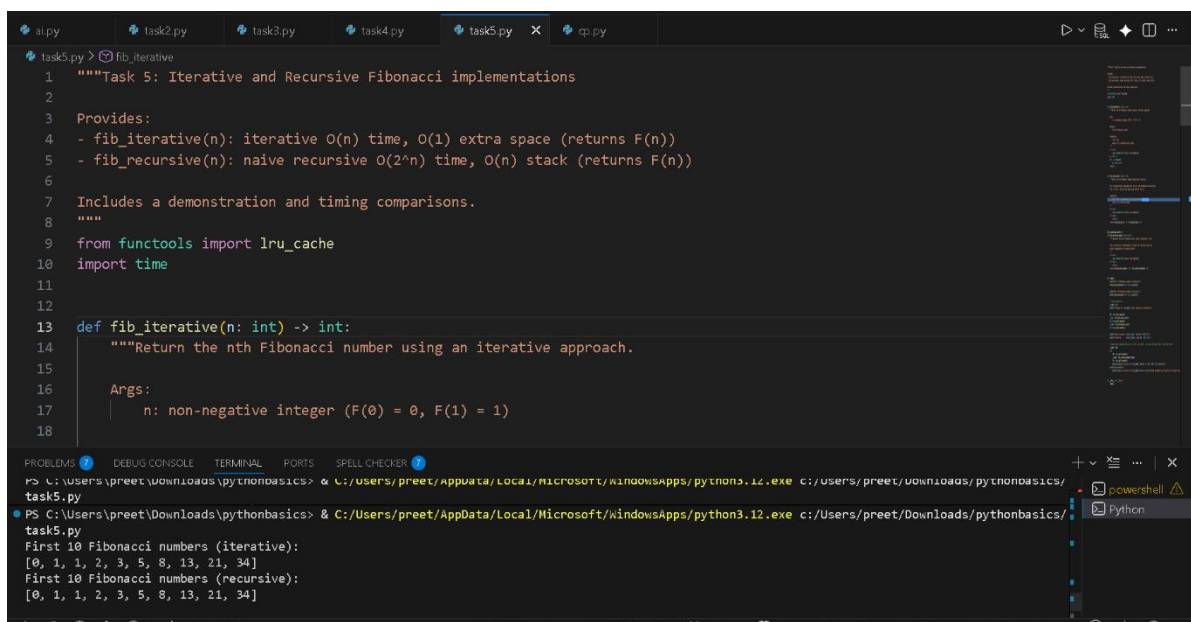
Prompt GitHub Copilot to generate:

An iterative Fibonacci implementation

A recursive Fibonacci implementation

❖ Expected Output

- Two correct implementations
- Explanation of execution flow for both
- Comparison covering:
 - Time and space complexity
 - Performance for large n
 - When recursion should be avoided



```
1 """Task 5: Iterative and Recursive Fibonacci implementations
2
3 Provides:
4 - fib_iterative(n): iterative O(n) time, O(1) extra space (returns F(n))
5 - fib_recursive(n): naive recursive O(2^n) time, O(n) stack (returns F(n))
6
7 Includes a demonstration and timing comparisons.
8 """
9 from functools import lru_cache
10 import time
11
12
13 def fib_iterative(n: int) -> int:
14     """Return the nth Fibonacci number using an iterative approach.
15
16     Args:
17         n: non-negative integer (F(0) = 0, F(1) = 1)
18
19     Returns:
20         The nth Fibonacci number.
21     """
22     if n < 0:
23         raise ValueError("n must be a non-negative integer")
24     if n == 0:
25         return 0
26     if n == 1:
27         return 1
28     a, b = 0, 1
29     for _ in range(2, n + 1):
30         a, b = b, a + b
31     return b
32
33 def fib_recursive(n: int) -> int:
34     """Return the nth Fibonacci number using a naive recursive approach.
35
36     Args:
37         n: non-negative integer (F(0) = 0, F(1) = 1)
38
39     Returns:
40         The nth Fibonacci number.
41     """
42     if n < 0:
43         raise ValueError("n must be a non-negative integer")
44     if n == 0:
45         return 0
46     if n == 1:
47         return 1
48     return fib_recursive(n - 1) + fib_recursive(n - 2)
49
50 if __name__ == "__main__":
51     # Demonstration and timing comparisons
52     n = 10
53     print(f"First {n} Fibonacci numbers (iterative):")
54     print(fib_iterative(0), fib_iterative(1), fib_iterative(2), fib_iterative(3), fib_iterative(4),
55           fib_iterative(5), fib_iterative(6), fib_iterative(7), fib_iterative(8), fib_iterative(9),
56           fib_iterative(10))
57     print(f"First {n} Fibonacci numbers (recursive):")
58     print(fib_recursive(0), fib_recursive(1), fib_recursive(2), fib_recursive(3), fib_recursive(4),
59           fib_recursive(5), fib_recursive(6), fib_recursive(7), fib_recursive(8), fib_recursive(9),
60           fib_recursive(10))
```

```
PS C:\Users\preet\Downloads\pythonbasics> & C:/Users/preet/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/Users/preet/Downloads/pythonbasics/task5.py
First 10 Fibonacci numbers (iterative):
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
First 10 Fibonacci numbers (recursive):
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

OUTPUT:

First 10 Fibonacci numbers (iterative):

[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

First 10 Fibonacci numbers (recursive):

[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

Timing for n=30 (naive recursive vs iterative):

naive recursive: value=832040, time=0.185535s

iterative: value=832040, time=0.000009s

memoized recursive for n=500 computed in 0.001015s (memoized)

CODE EXPLANATION

Task 5: Iterative vs Recursive Fibonacci

fib_iterative(n)

- Uses a loop to calculate Fibonacci
- Updates values using $a, b = b, a + b$
- Time: $O(n)$
- Space: $O(1)$
 - ✓ Fast and memory-efficient

fib_recursive(n)

- Follows the formula $F(n) = F(n-1) + F(n-2)$
- Repeats many calculations
- Time: $O(2^n)$
- Space: $O(n)$ (call stack)
 - ✗ Very slow for large n

fib_recursive_memo(n)

- Uses @lru_cache to store previous results
- Avoids repeated work
- Time: $O(n)$
- Space: $O(n)$
 - ✓ Fast but still uses recursion

Demo & Timing

- Prints first 10 Fibonacci numbers
- Compares execution time of iterative vs recursive
- Shows memoized recursion is much faster than naive recursion

Conclusion

- Iterative → best for performance

- Naive recursive → only for learning
- Memoized recursive → good balance, but iterative is safest for large n